# WE'RE STARTING AT 9, IF YOU WANT A HOSTED LAB SEE US AT THE FRONT OF THE ROOM & WE'LL DO OUR BEST TO HELP

## If you did the hosted lab

- What you should do right now is log back in to the AWS console you were given access to via email to ensure you can reach it. If you cannot for any reason, please raise your hand when you know that.
- If you requested a lab but never heard back, see us in the front.
- If you  received a lab but never logged in, log in now following the email instructions.
- If you got email instructions but not all the pieces, see us in the front.
- AWS ACCT ID: 213295720664

## If you opted to self host

- You have chosen the harder path
- You need to get to work prepping your system right now while we do the starting slides:

# MCP Hackathon: Master Your AI's Identity

A 42 Notions Joint

By Sander @ 42 Notions && Steele @ OpenAI

# Some Logistics

## If you did the hosted lab

- We're going to walk through that with you here step by step.
- What you should do right now is log back in to the AWS console you were given access to via email to ensure you can reach it. If you cannot for any reason, please raise your hand when you know that.
- Once you're logged in, then wait for further steps. Note that AWS is aggressive about timeouts and you may be kicked out and need to log back in later.

## If you opted to self host

- You have chosen the harder path
- You need to get to work prepping your system right now while we do the starting slides:

# Some Logistics


## If you did the hosted lab

- BUT you did not log in to AWS and change you password, then you have to do that now.
- You need two emails:
  - The one from secure@onetimesecret.com which has the link to the vault service to retrieve your personalized initial password
  - The one from sander@42notions.com with subject “Identiverse 2026 MCP Hackathon - Personal Lab Details” which has the passphrase you need to use with the vault service (as well as all the rest of the log on details)
- Then you need to change your password on first login to the AWS console

# What is the point of this workshop?

- IAM is one of the few controls that can have an effect on AI agents right now.
- Most identity professionals are not being given the insight into the places where the opportunities to affect things are happening.
- People building AI agents are doing so using tools and descriptions that are hard to get a handle on because of the pace that development is happening.
- Our goal today is to give you a guide to the most important places where IAM and AI agent deployments are intersecting from as much of hands-on point of view as possible.

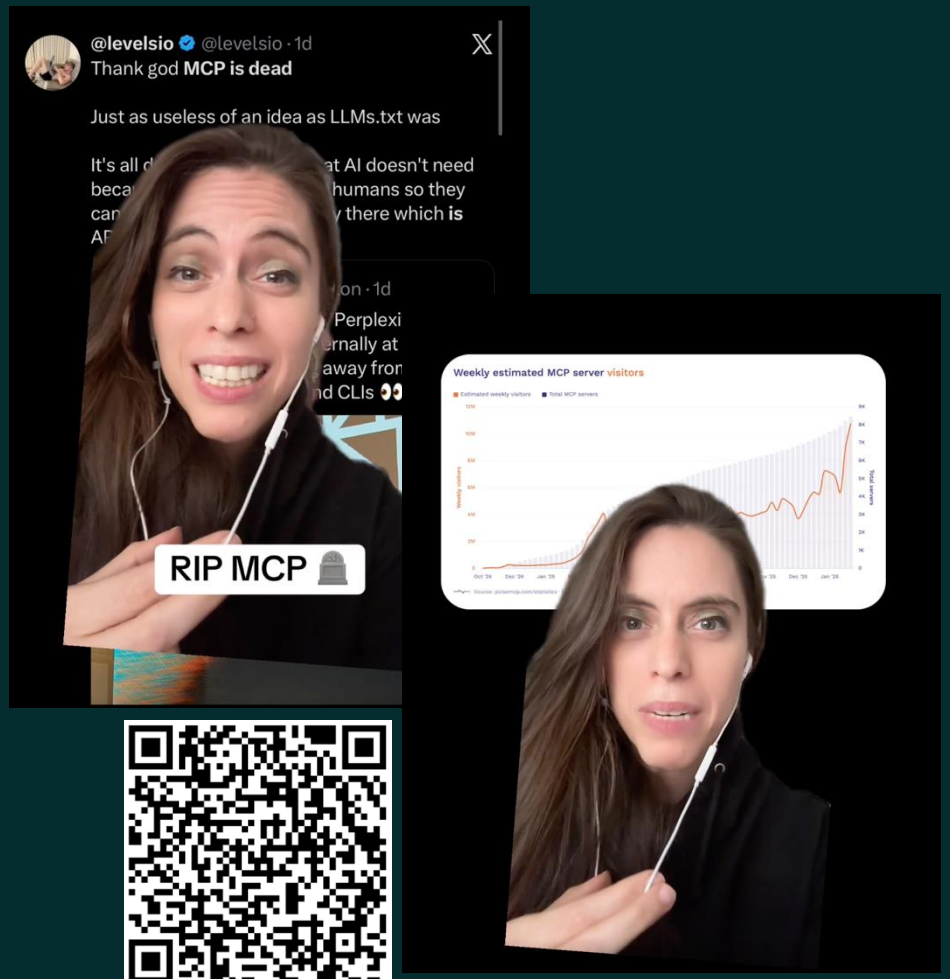
# MCP?

- The Model Context Protocol (MCP) is an open standard released by Anthropic late November 2024 for connecting AI assistants to the systems where data lives (*e.g.*, content repositories, databases, and dev environments) with the aim of helping frontier models produce better, more relevant responses. It is currently maintained by the Agentic AI Foundation under the Linux Foundation.
- It's crucial to understand that MCP was built from the point of view of the agentic AI developers. The goal was to get the data and integrations these systems needed to these LLM powered systems as cleanly as possible and quickly as possible.
- Of course, we can all guess what that meant for the use of proper controls and IAM practices…

Agentic AI Foundation

THE LINUX FOUNDATION

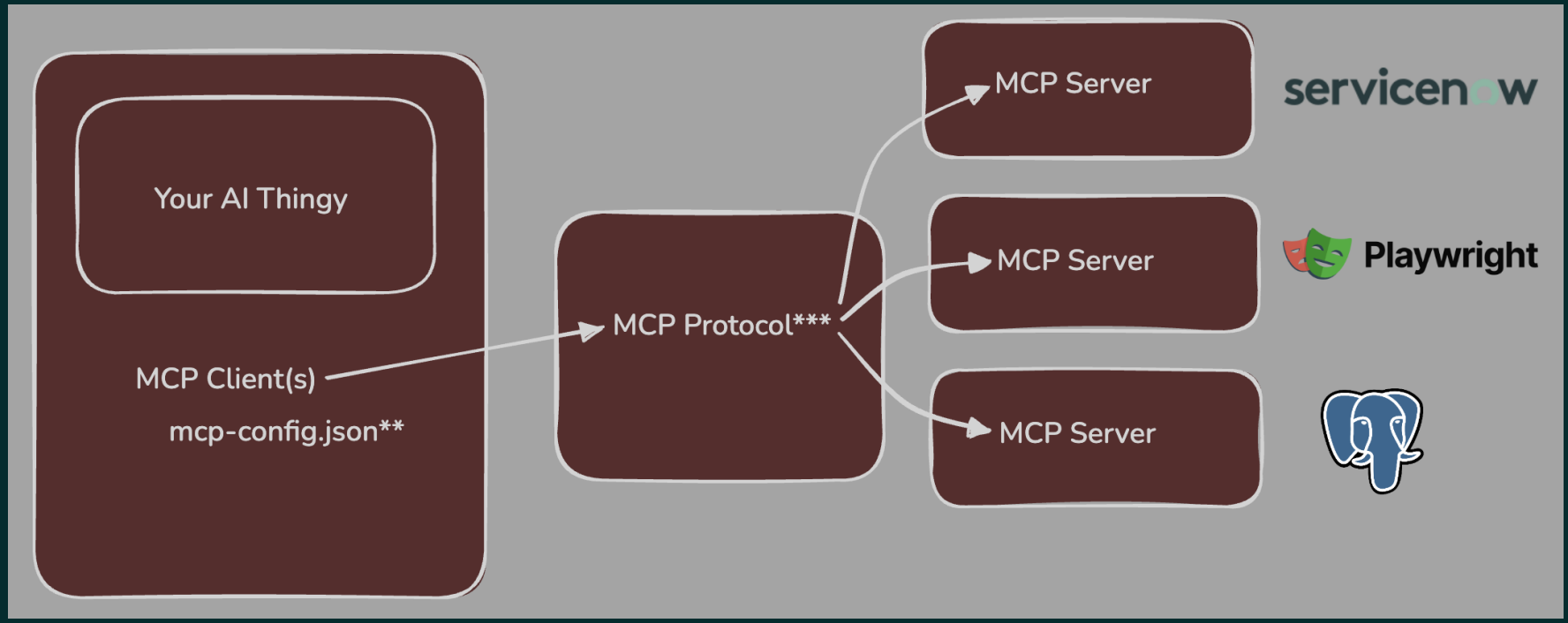
# Isn’t MCP dead already?




- Anything that touches AI is news, anything Anthropic or other foundation players do is news. anything that has “died” is news, and so anything that was made by Anthropic for use by AI that has died is big news. Even if it’s not true.
- What is true is that MCP had a huge rush of interest and adoption when it was announced which cooled off quickly when devs realized it was heavy for their needs.
- Many then started to turn back to direct use of APIs.
- For an IAM pro, this pattern should feel familiar. It’s easier for the developer to just use the native cloud service API versus the organization’s approved driver that builds in all the controls, auditing and other governance and observability – right? That’s basically the story of MCP so far.

# MCP components

- Model Context ***PROTOCOL***
- There's no current requirements for how you build the clients or servers.
- You can find MCP clients & servers written in every modern language
- The only true requirement is implementing the protocol



# MCP components

```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": [
        "@modelcontextprotocol/server-filesystem",
        "./"
      ]
    },
    "webresearch": {
      "command": "npx",
      "args": [
        "-y",
        "@mzxrai/mcp-webresearch"
      ]
    }
  },
  "ollama": {
    "host": "http://localhost:11434",
    "model": "qwen2.5:latest"
  }
}
```

MCP Server

- Actual server
- Entry in the MCP client configuration

MCP Client

- What your AI app touches

# MCP is more than tools

Thanks Claude!

**Server-exposed primitives:**

**Resources** are read-only context the server exposes, addressed by URI (file://…, custom schemes). Unlike tools, they're *application-controlled* — the host app decides when to pull them into context, not the model. Each has a URI, name, MIME type, and content. Servers can also expose **templates** (db://users/{id}) the client expands, and clients can subscribe to changes. Use them for "data the model may look at," not actions.

**Prompts** are user-controlled templates — reusable, parameterized message sequences, usually surfaced as slash commands. The user picks one, supplies arguments, and the server returns ready-to-send messages. They package repeatable workflows.

So the server side is a triad: **tools** (model invokes), **resources** (app includes), **prompts** (user invokes).

**Client-exposed capabilities** (the server can call back into these):

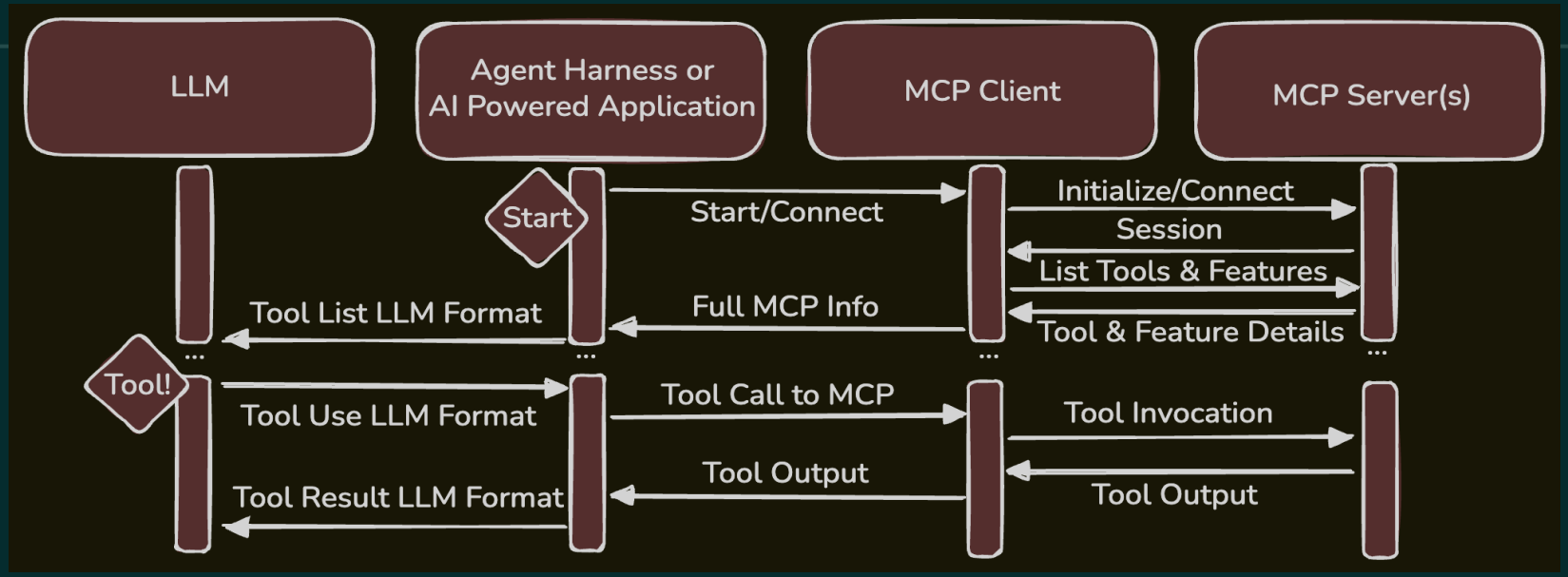
**Sampling** lets a server request an LLM completion from the client, borrowing its model instead of holding an API key — with a human approval step expected.

**Roots** are the filesystem/URI boundaries the client declares to the server: "operate only within these scopes."

**Elicitation** lets a server pause and request structured input from the user (via a JSON Schema) for a missing parameter or confirmation, instead of failing.

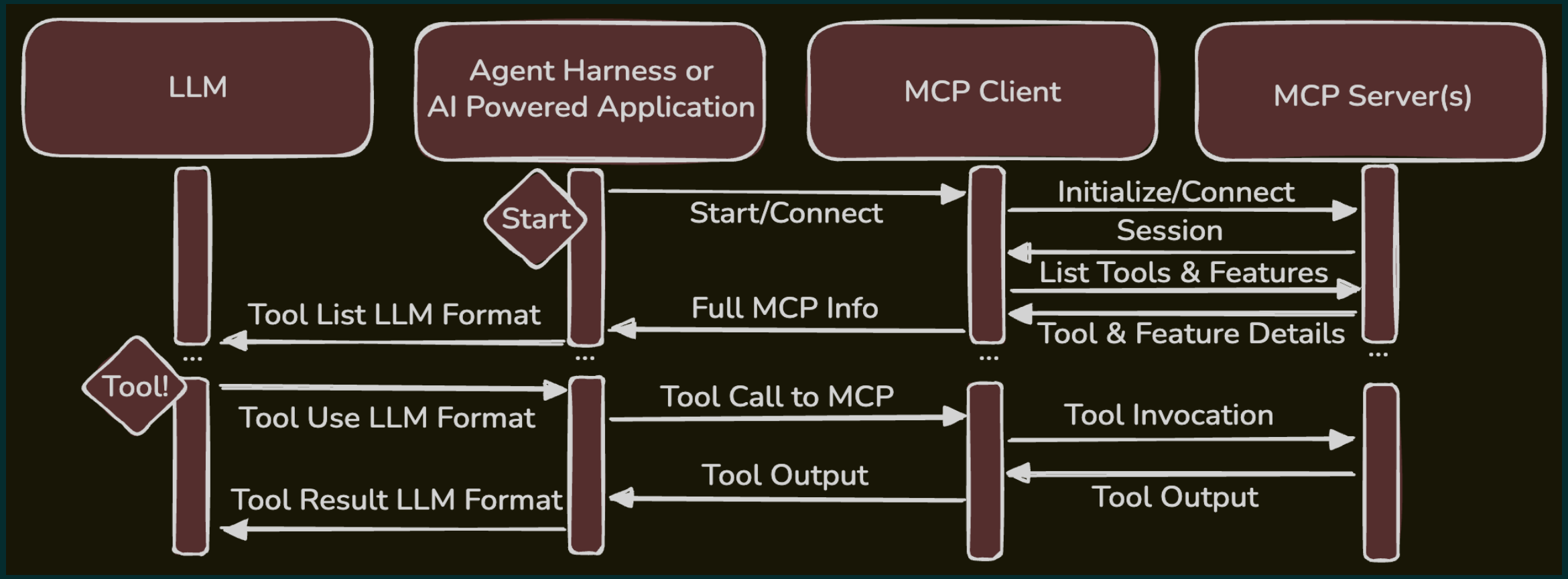
**Plumbing worth knowing exists:** capability negotiation at initialize (each side declares what it supports), listChanged and progress **notifications**, structured **logging**, argument **completion**, and **transports** (stdio locally, Streamable HTTP remotely) — all over JSON-RPC 2.0.

# MCP components




1. At startup, the agent harness, LLM powered application, or other system using MCP calls the MCP client to connect to each configured MCP server and query what it exposes, most often seen as a list of tools available and their properties.
2. This information is then formatted into the model's native tool definition format to be injected into the context along with all other prompt information (system prompt, user prompts, etc.).
3. When in the processing of some prompt the LLM decides that a tool would be useful, a tool_use call is made in the LLM's native format, and that's translated into MCP by the MCP client to be executed.
4. The MCP client calls the MCP Server through the MCP protocol, the tool is executed using whatever opaque methods the server is built to use, and results come back.
5. The MCP tool output is translated to the model's native tool_result message format, and that is appended to the prompt with all other relevant results by the harness or application operating the LLM.
6. The LLM evaluates the results, and may choose to make more tool calls or move on processing as it generates the response.

# MCP components

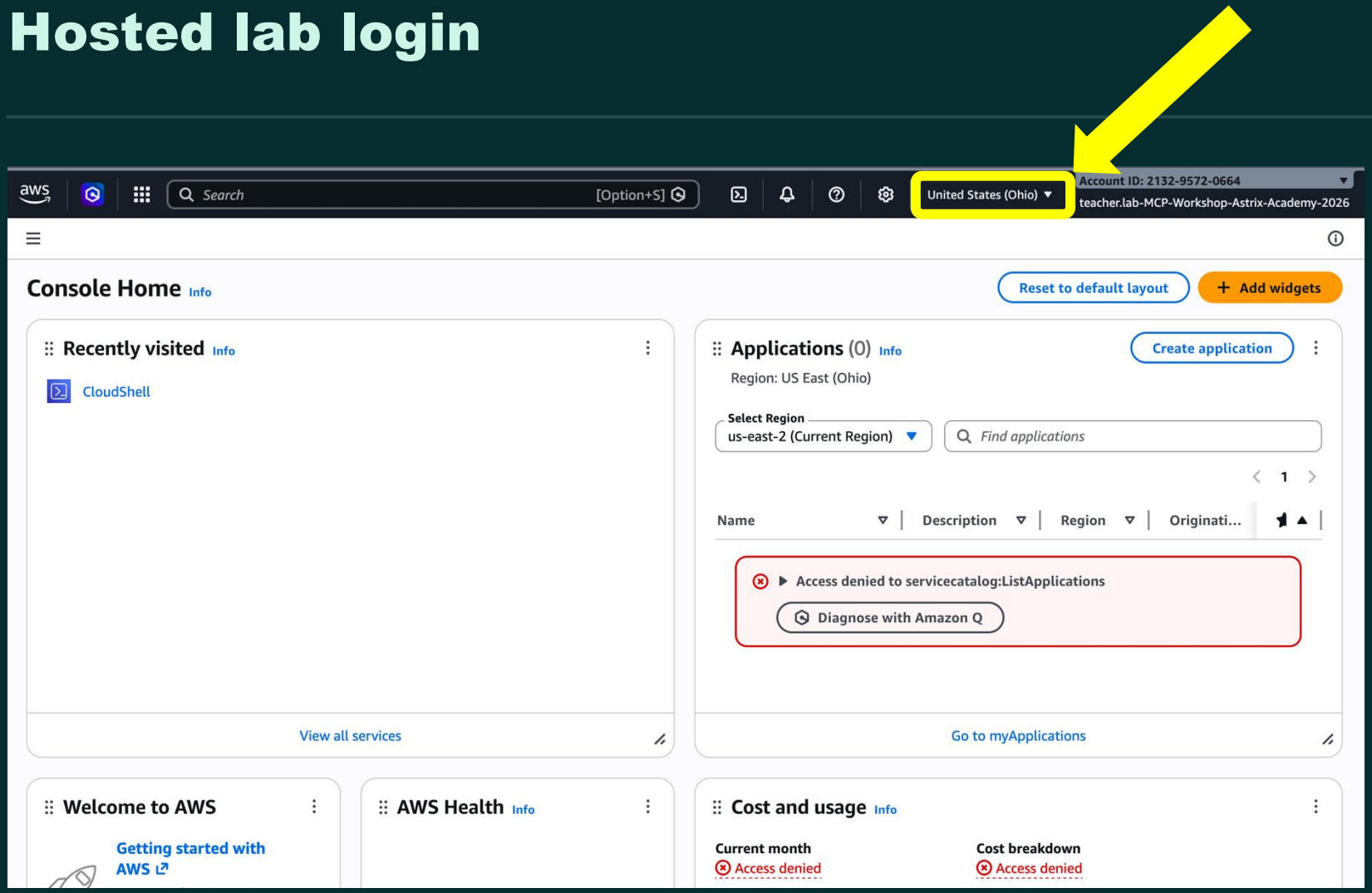


# Let’s start labbing…

# Some notes about our lab systems

- These labs are not pristine. They have been optimized to be low cost to run, fit on small hardware, but be complete from a teaching point of view.
- Things will go wrong, and that is part of what we're trying to do here. So be extra careful to follow the directions so you can tell the difference between planned failure and user error.
- Rush ahead at your own peril. You may execute all the steps as planned and hit a planned blocker, but you'll need to wait for all of us to catch up to find out why.

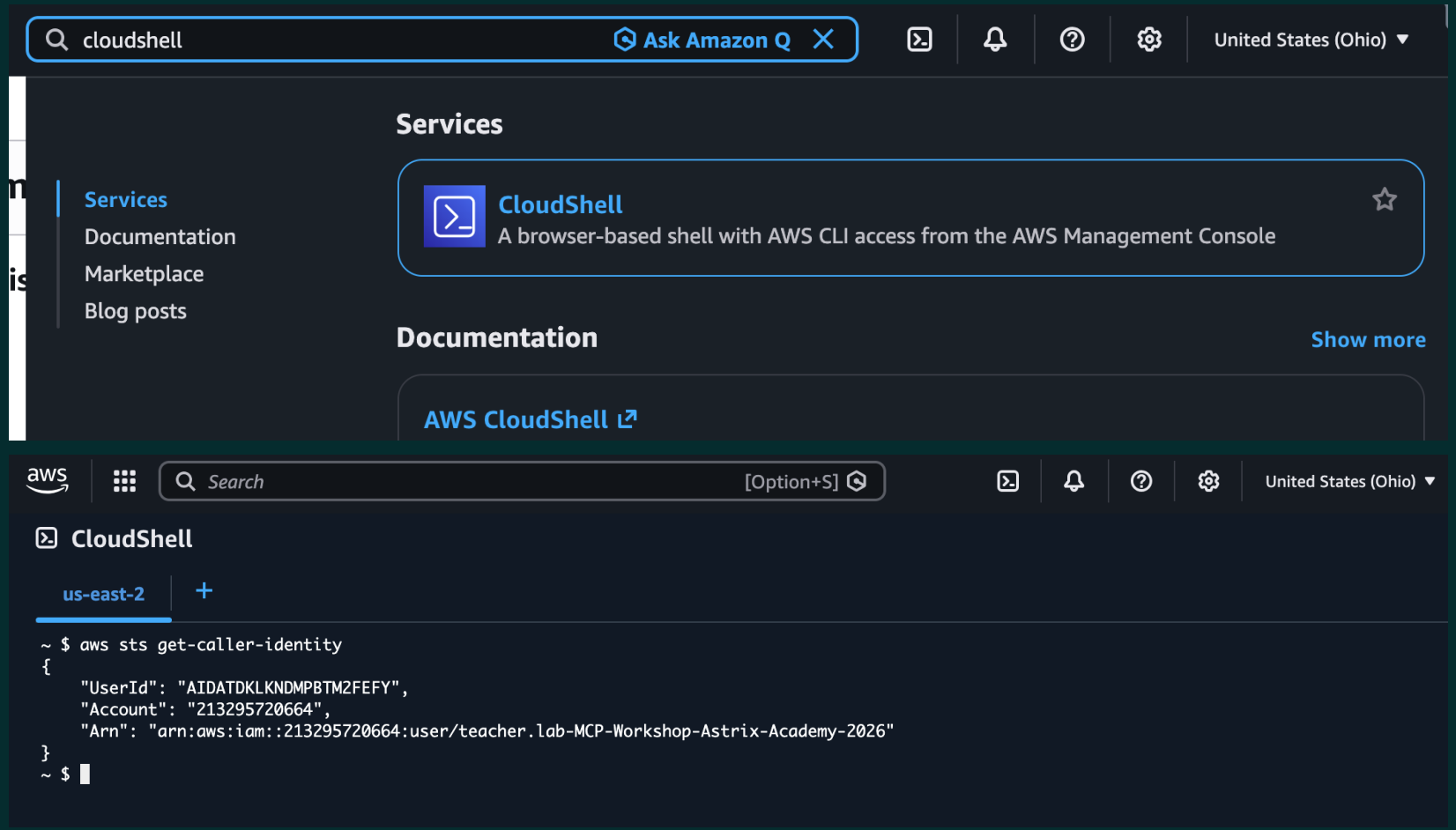
# Hosted lab login




- If you see a screen that looks like this (possibly with an empty “Recently visited” widget), then you’re in good shape.
- Make sure it’s set to “United States (Ohio)” (us-east-2) for the region, which is highlighted in yellow in the screen here

# Hosted lab first steps

- Launch the CloudShell
- Make sure the CloudShell is in us-east-2 by check the title at the top of the tab of the launched shell
- Then we want to be sure you have a good session (AWS CloudShell sometimes gets… confused)
- But before you start typing…



# The github repo with all the toys

**This repo has everything you need**

- https://github.com/sanderiam42/MCP-Workshop-IDV-Prime-2026

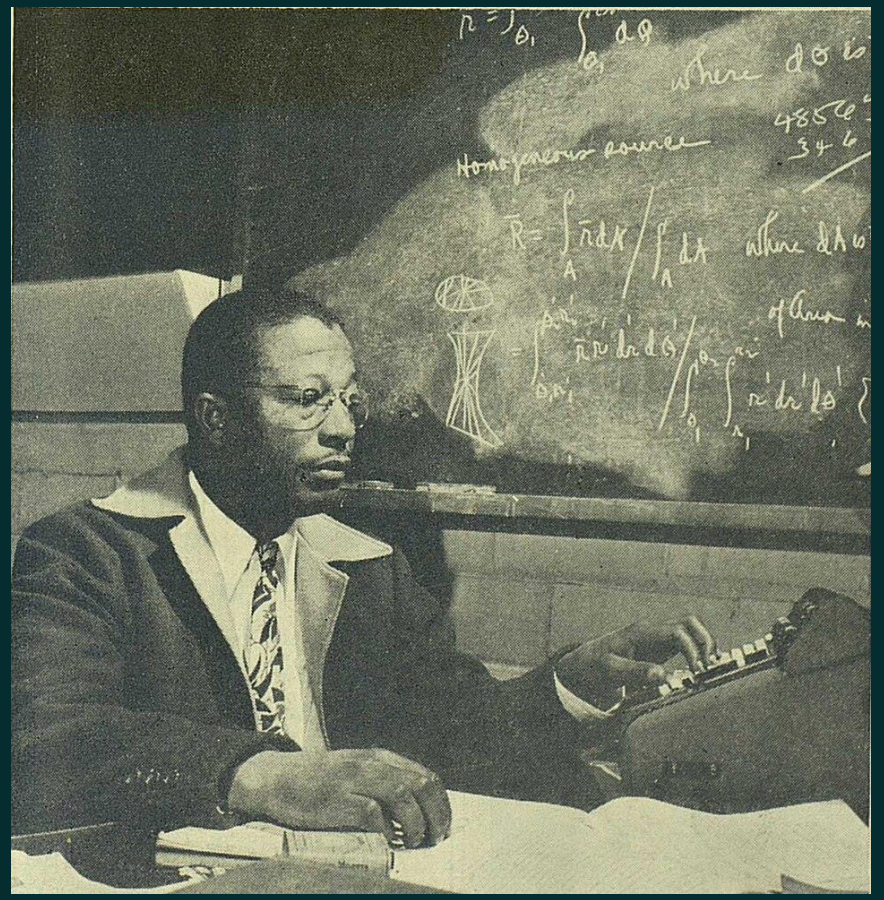
- cheatsheet/copyPasta.txt

- DO NOT START STEPS YET PLEASE
- YOU CAN ONLY COPY ONE LINE AT A TIME FROM copyPasta.txt

# Lab Set Up Check In (or break if you're all set)

# Lab 1: Basic MCP Tool Use

# Follow along in docs & demo



# Review what Lab 1 showed us

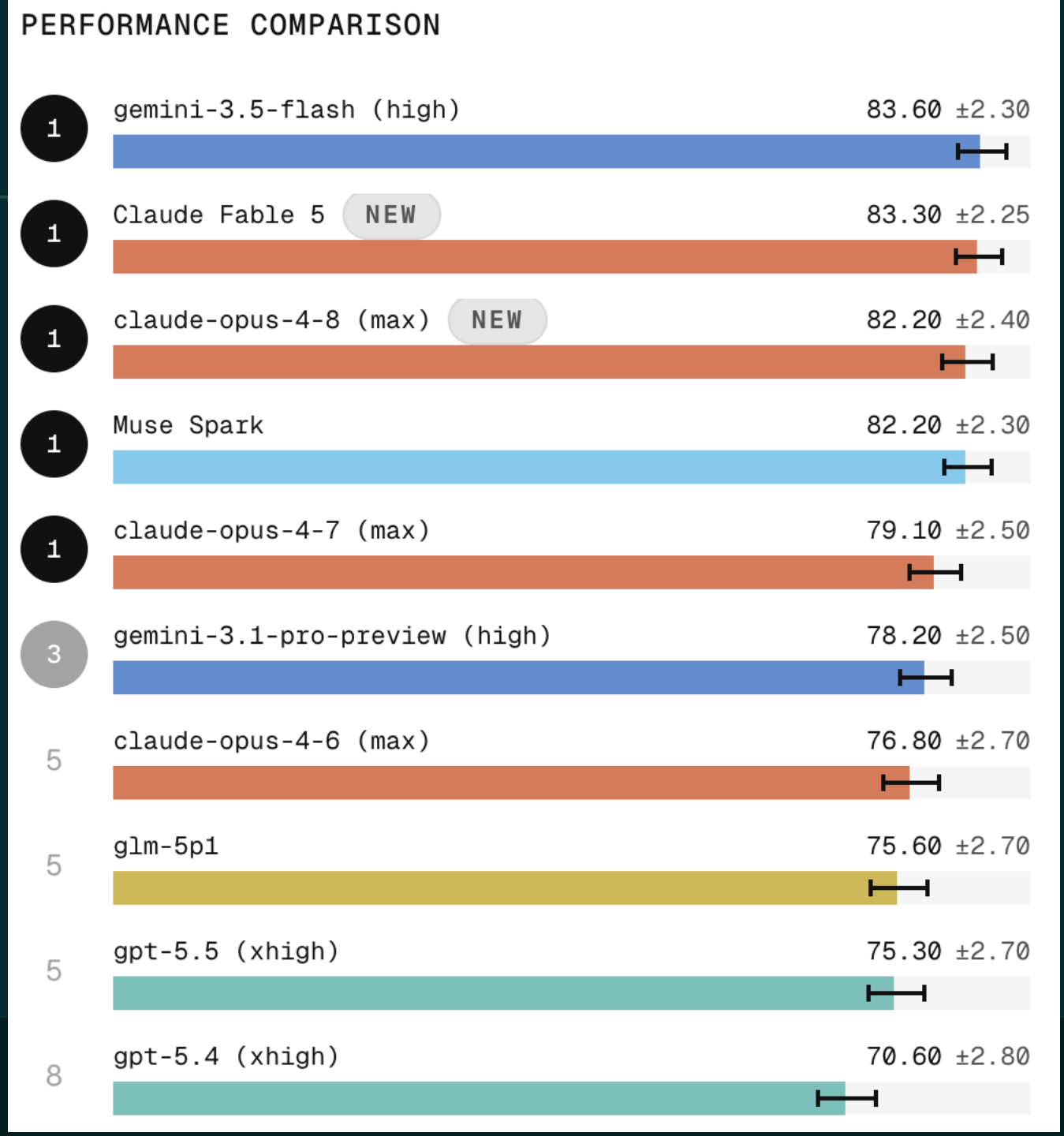
- The non-determinism is real.
  - It’s very easy to get used to how smart ChatGPT or Claude can be. Smaller models used in specialized applications may not have those capabilities and design must deal with that.
- MCP configs can have very toxic content.
  - Obviously, hard coded credentials are bad.
  - You can’t rely on logging alone. There can be credentials in configs that may never get used which are pulled from repos.
- The layers in these stacks get deep.
  - It’s not unusual for there to be as many layers as these labs have – and even more.
  - There are also a lot of gaps in people’s understanding of how the parts work, how they relate to controls like A&A, and where governance and observability can even be applied.
- The LLM may affect tool use.
  - Not every LLM’s tool use is created equal, and LLMs may need more help and may encourage bad behavior.

# The LLM may affect tool use

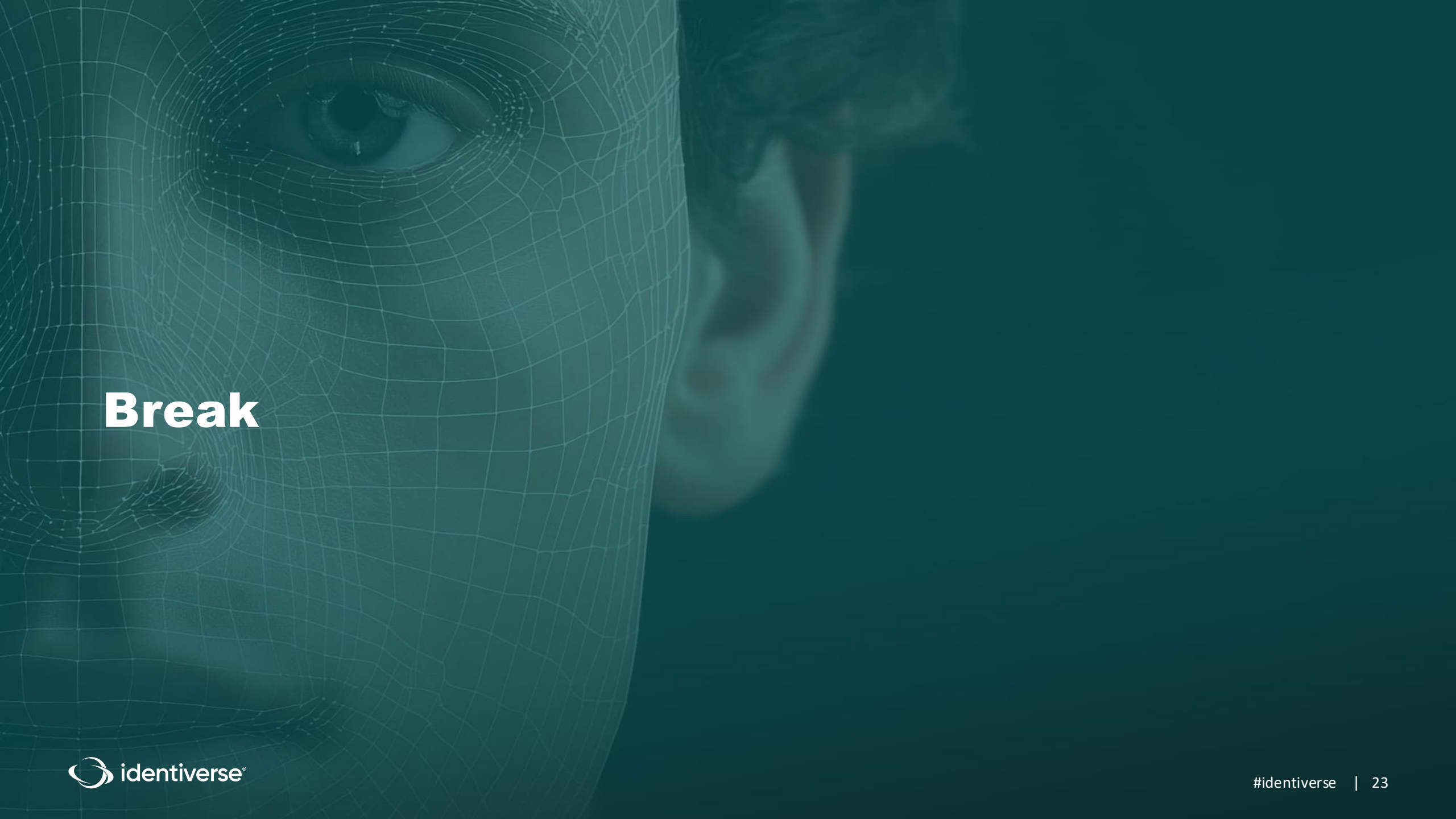
scale labs

## MCP Atlas

Evaluating real-world tool use through the Model Context Protocol (MCP)




identiverse

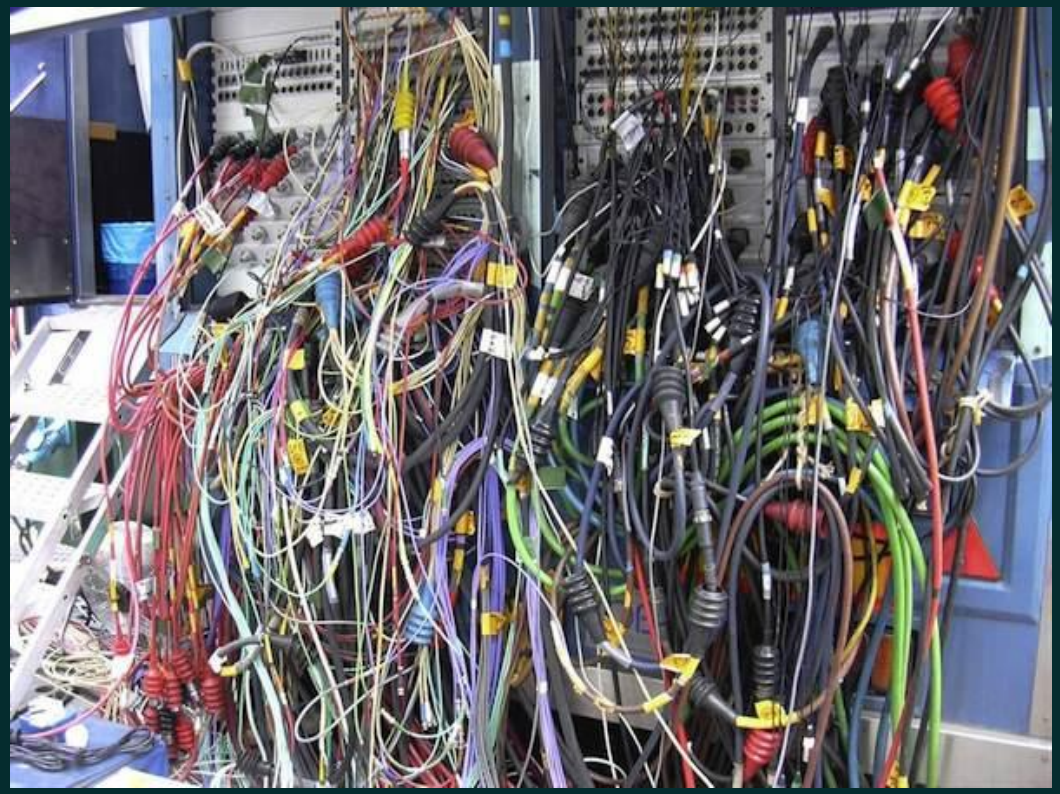


# Break

# Lab 2: Securing MCP Configs

# Follow along in docs & demo

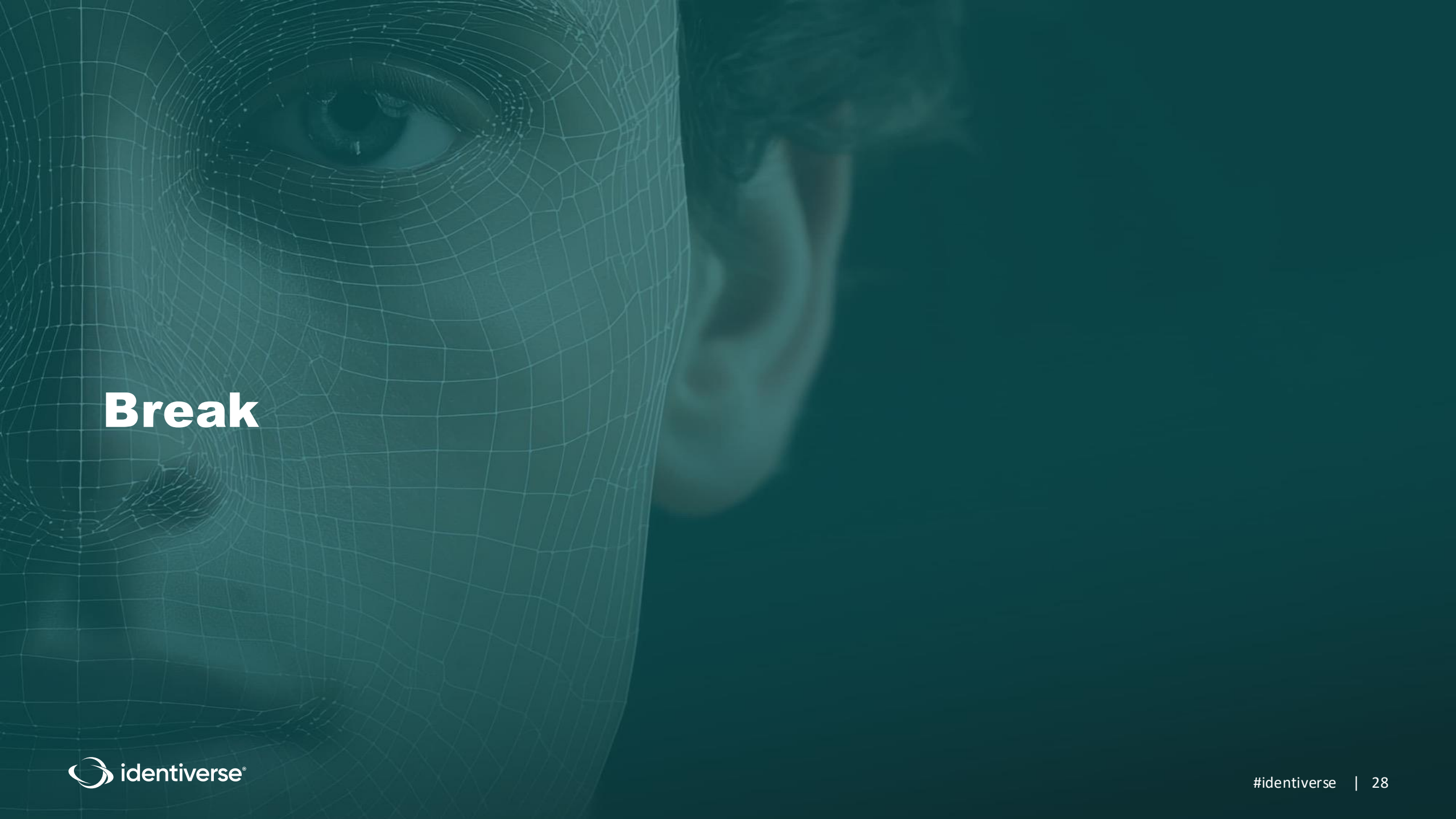
# MCP transport types



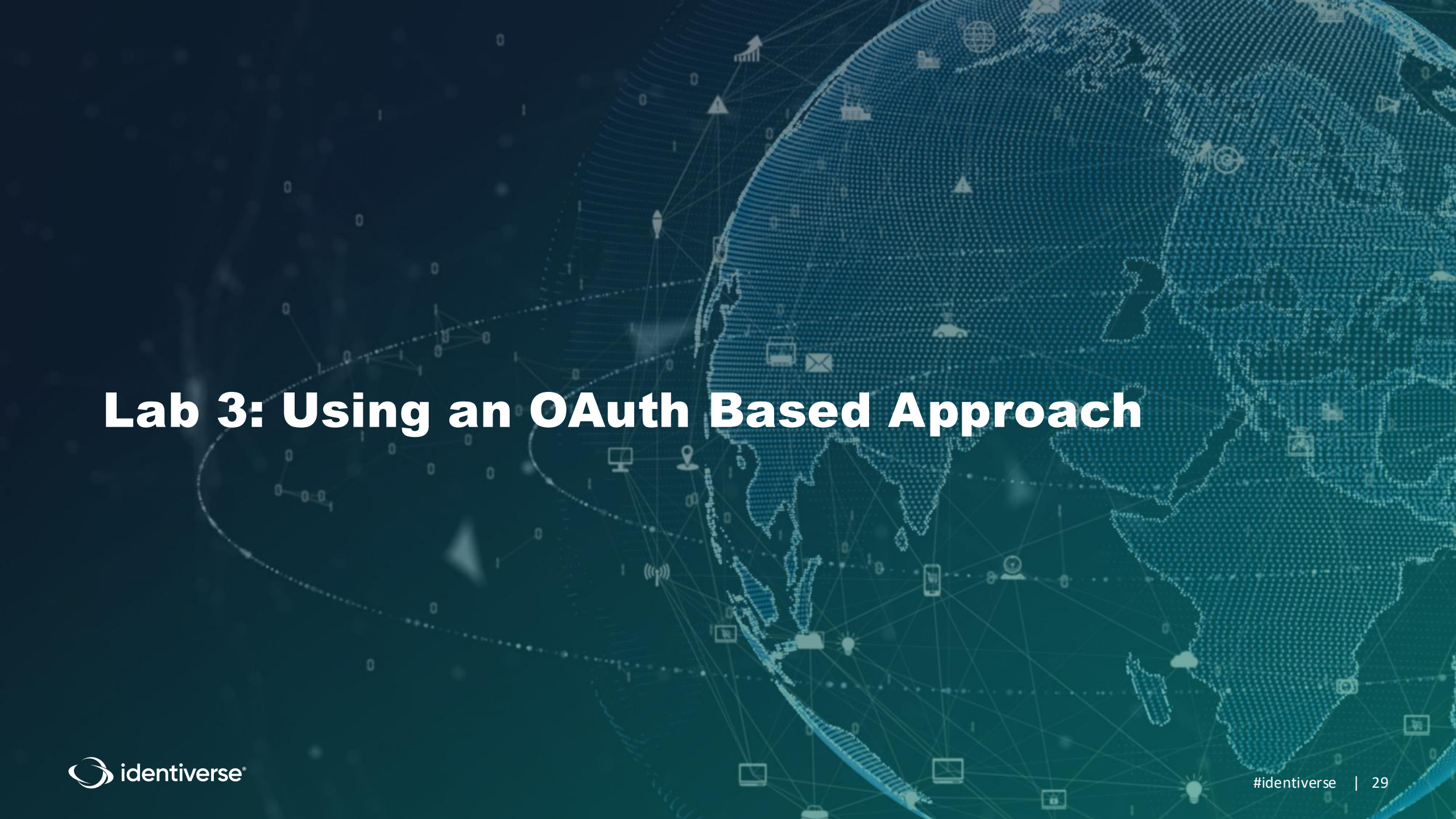
- The MCP protocol does not care how the client and server communicate with one another
  - This is partially why so many think of the “MCP Server” as being part of the local application stack. When using a stdio transport, it basically is. The “server” is a local process and the communications are all over local memory (the client writes JSON-RPC to the server's stdin and reads from its stdout).
  - This allows devs to treat any MCP server the same regardless of where it lives. If the credentials needed pass over a network or only through the local machine, it’s all the same to the MCP config.

# Review what Lab 2 showed us

- MCP transport type has security effects.
  - Even though there's a big difference between communications over standard input & output versus streaming https, the config in the MCP side hides this.
  - People may not even realize what information is flowing where. Even if it's a "command" that command may not be running fully local.
- Vaulting will require some bootstrapping.
  - AWS gave us built in machine identity and user principals, but that won't always work.
  - SPIFFE and similar approaches can work, but have high complexity.
- Many don't understand MCP flows.
  - People call it an "API wrapper" but fail to see how the pieces fit together.
  - When and how MCP will connect, carry sessions, and use the IAM assets it has need to be understood if the behavior is to be effectively controlled and monitored.
  - This is part of what fuels calls to just use regular APIs, but those will then be subject to the non-deterministic tax through random calls and uses of these IAM assets.



# Break



# Lab 3: Using an OAuth Based Approach

# So much AuthN & AuthZ work going on…

Thanks Claude!

| Dimension | XAA / ID-JAG | AAuth | AuthZed / SpiceDB |
|---|---|---|---|
| AuthN | Consumes enterprise IdP's authentication | Gives agents own cryptographic identity | Agnostic; done elsewhere |
| AuthZ | Brokers delegated cross-app access | Carried in token, per-call | Its entire job; fine-grained |
| Crosses trust domains? | Yes — its core purpose | Yes — built for it | No — central decision point |
| Most like (in wide use) | OAuth token exchange; SAML federation | SPIFFE plus signed HTTP requests | Google Zanzibar (its blueprint) |
| Biggest red flag | Still ends in bearer tokens | Immature, early single-author draft | Central dependency you must operate |

# Follow along in docs & demo

# Review what Lab 3 showed us

- OAuth may not be our savior.
  - …but it does have the biggest lead as things stand right now.
  - There will still be secrets in config files using OAuth even at the bleeding edge.
  - We want SPIFFE like self-contained verifiability, cross domain portability, and fine grained delegation all in one.


- Configs for MCP/APIs and OAuth are hard.
  - Devs will absolutely not wait to do this “the right way” given the current state of things. The enforcement will chafe board level timeline dictates to “AI ALL THE THINGS” and we will be the bad guys slowing things down (again).
  - If your argument is “this gets rid of secrets” get ready for smart devs to fight you about it.

- The paint is still wet.
  - Nothing in this space is fully ready to deploy at most mature shops.

# We did it!

# So what now?

- Hosted labs will be destroyed in the next 24 hours or sooner. Anything you want to extract from them you should do ASAP.
- The repo with all the artifacts will stay up & public as long as possible.

- What can you do with the knowledge you've gained:
  1. Dissect any LLM stuff you have access to and find the MCP config – even if you're not using that in the tool. Got ChatGPT or Claude desktop apps? Using one of the "claws"?
  2. Ask your AI folks probing questions about how they built their tool interactions, where the identity assets came from and get stored, what their plans around for MCP. Did they reject MCP for API? Why?
  3. Perform a readiness assessment for tool use, or a threat model if it's in use already. IdP(s) ready for this? Vault(s)? Relying parties?

Lab Repo

https://github.com/sanderiam42/MCP-Workshop-IDV-Prime-2026

# Ask us anything

Us


Hopefully not you

Wait? You don't know this Monty Python sketch?!?

This takes you the sketch. But watch the whole movie!

# Thank You!